

6-DoF Robotic Arm Simulation

ROS 2 Jazzy + UR5e + RViz2 + Gazebo Harmonic

1 Foreword

This document has been formatted using ChatGPT. The process, matter and algorithms are all created through genuine study and effort.

Find the project here on github -

[https://github.com/BoldRhythm/
UR5e-6DoF-Robotic-Arm-sim-ROS2-Jazzy-Rviz2-ROS2-Jazzy-Gazebo-Harmonic-/tree/main](https://github.com/BoldRhythm/UR5e-6DoF-Robotic-Arm-sim-ROS2-Jazzy-Rviz2-ROS2-Jazzy-Gazebo-Harmonic-/tree/main)

2 Objective

Note: This has been developed on native Ubuntu 24.04. To make things more accessible, using WSL2 on Windows is much easier.

The goal of this project is to set up and control a 6-DoF UR5e robotic arm using ROS 2, RViz2, Gazebo Harmonic, TF2, and `ros2_control`.

The implementation is divided into two stages:

1. **RViz2:** build and verify the ROS 2 side of the robot.
2. **Gazebo Harmonic:** use the working ROS 2 setup with a simulated robot and controllers.

RViz2 is implemented first because it is simpler. It allows the robot description, joint states, TF2, and ROS 2 communication to be checked before simulation is introduced.

Gazebo is then added as the simulation layer.

3 Software and Installation

The setup uses:

- Ubuntu 24.04
- ROS 2 Jazzy
- Python 3
- RViz2
- Gazebo Harmonic
- TF2 and RQt TF Tree
- `ros2_control`
- Universal Robots UR5e description

3.1 ROS 2 Jazzy

Use the official ROS 2 Jazzy installation guide:

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

After installation:

```
source /opt/ros/jazzy/setup.bash
```

Check:

```
ros2 --help
```

3.2 Required Packages

Install the packages used in the project:

```
sudo apt update

sudo apt install \
ros-jazzy-rviz2 \
ros-jazzy-robot-state-publisher \
ros-jazzy-xacro \
ros-jazzy-joint-state-publisher-gui \
ros-jazzy-tf2-tools \
ros-jazzy-rqt-tf-tree \
ros-jazzy-ros2-control \
ros-jazzy-ros2-controllers \
ros-jazzy-ros2controlcli \
ros-jazzy-ur-simulation-gz
```

4 Create the Workspace

Create the ROS 2 workspace:

```
mkdir -p ~/Projects/ros2_6DoF/src
cd ~/Projects/ros2_6DoF

source /opt/ros/jazzy/setup.bash
```

Clone the official Universal Robots description:

https://github.com/UniversalRobots/Universal_Robots_ROS2_Description

Use the Jazzy branch:

```
cd ~/Projects/ros2_6DoF/src

git clone -b jazzy \
https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.git \
ur_description
```

Build and source the workspace:

```
cd ~/Projects/ros2_6DoF  
  
colcon build  
source install/setup.bash
```

5 Stage I – UR5e in RViz2

This section deals only with the RViz2 implementation.

RViz2 is used first because it provides a simple way to check the ROS 2 side of the robot without introducing simulation or controllers.

The basic structure is:

```
URDF / Xacro
  |
  v
Robot Description
  |
  v
robot_state_publisher
  |
  v
TF2
  |
  v
RViz2
```

5.1 Launch the UR5e

Launch the robot description:

```
ros2 launch ur_description view_ur.launch.xml ur_type:=ur5e
```

If required, the launch file can be run directly:

```
ros2 launch \
~/Projects/ros2_6DoF/src/ur_description/launch/view_ur.launch.xml \
ur_type:=ur5e
```

The UR5e should appear in RViz2.

Now you can see the available topics, which will be used to issue commands to our simulation. The key commands to inspect any topic are -

```
ros2 topic list
ros2 topic info <topic_name> -v
ros2 interface show <name you see in the topic info output>
```

Gazebo is not used at this stage.

5.2 URDF and Xacro

The robot is described using URDF and Xacro.

- **URDF** describes links, joints, geometry, and frames.
- **Xacro** makes the robot description easier to manage.

The basic process is:

Xacro → Robot Description

The six controllable UR5e joints are:

```
shoulder_pan_joint  
shoulder_lift_joint  
elbow_joint  
wrist_1_joint  
wrist_2_joint  
wrist_3_joint
```

5.3 TF2

TF2 manages transformations between coordinate frames.

Check the transformation from the base to the tool:

```
ros2 run tf2_ros tf2_echo base_link tool0
```

The output contains translation and rotation information.

The TF tree can be viewed using:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

If required:

```
ros2 run rqt_tf_tree rqt_tf_tree --force-discover
```

The main frame chain is:

```
base_link  
|  
shoulder_link  
|  
upper_arm_link  
|  
forearm_link  
|  
wrist_1_link  
|  
wrist_2_link  
|  
wrist_3_link  
|  
tool0
```

5.4 Quaternions

ROS uses the quaternion ordering:

$$(x, y, z, w)$$

For a rotation θ around the x axis:

$$q = \left(\sin \frac{\theta}{2}, 0, 0, \cos \frac{\theta}{2} \right)$$

In a ROS message:

```
orientation.x
orientation.y
orientation.z
orientation.w
```

Always check the quaternion convention when working with another library.

5.5 Joint States

The six joint names must be kept in the same order as their position values:

```
joint_names = [
    'shoulder_pan_joint',
    'shoulder_lift_joint',
    'elbow_joint',
    'wrist_1_joint',
    'wrist_2_joint',
    'wrist_3_joint'
]

positions = [
    0.0,
    -1.57,
    0.0,
    0.0,
    0.0,
    0.0
]
```

All six joints should be included when publishing the joint state.

An incomplete joint state can result in an incomplete robot state or TF chain.

Check the topic:

```
ros2 topic echo /joint_states
```

6 ROS 2 Python Node

Once the robot works in RViz2, a Python node can be introduced.

The basic node structure is:

```
import rclpy
from rclpy.node import Node

class UR5e(Node):

    def __init__(self):
        super().__init__('ur5e')

def main(args=None):
    rclpy.init(args=args)

    node = UR5e()
```

```

    rclpy.spin(node)

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

The class contains publishers, subscribers, timers, and callbacks.

The `main()` function starts ROS and keeps the node running.

6.1 Subscribers

A subscriber receives messages from a ROS 2 topic:

```

self.subscription = self.create_subscription(
    PoseStamped,
    '/goal_pose',
    self.pose_cb,
    qos
)

```

The callback processes the message:

```

def pose_cb(self, msg):

    x = msg.pose.position.x
    y = msg.pose.position.y
    z = msg.pose.position.z

    self.get_logger().info(
        f'Position: {x}, {y}, {z}'
    )

```

Before using an unfamiliar message, inspect it:

```

ros2 interface show geometry_msgs/msg/PoseStamped

```

6.2 QoS

ROS 2 uses Quality of Service settings to control communication.

A basic profile is:

```

from rclpy.qos import (
    QoSProfile,
    QoSReliabilityPolicy,
    QoSHistoryPolicy,
    QoSDurabilityPolicy
)

qos = QoSProfile(
    reliability=QoSReliabilityPolicy.RELIABLE,
    durability=QoSDurabilityPolicy.VOLATILE,
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=5
)

```

)

The important settings are:

- Reliability
- Durability
- History
- Queue depth

The publisher and subscriber need compatible QoS settings.

7 Stage I Checkpoint

Before moving to Gazebo, verify:

- The UR5e appears correctly in RViz2.
- All six joints are present.
- `/joint_states` contains the required joints.
- `base_link` to `tool0` exists in TF2.
- The TF tree is connected.
- The ROS 2 Python node can run correctly.

If these work, the ROS 2 side is ready for Gazebo.

8 Stage II – UR5e in Gazebo Harmonic

This section deals with the simulation and control side.

Gazebo is introduced only after the RViz2 implementation works.

The main difference is that Gazebo now provides a simulated robot whose joints can be controlled.

```
ROS 2 Node
  |
  v
ros2_control
  |
  v
Controller
  |
  v
Gazebo Harmonic
  |
  v
Simulated UR5e
```

8.1 Install the Gazebo Integration

Install the Universal Robots Gazebo package:

```
sudo apt install ros-jazzy-ur-simulation-gz
```

Check:

```
ros2 pkg list | grep ur_simulation_gz
```

8.2 Launch Gazebo

Launch the UR5e:

```
ros2 launch ur_simulation_gz ur_sim_control.launch.py \
ur_type:=ur5e launch_rviz:=false
```

Here:

- `ur_type:=ur5e` selects the UR5e.
- `launch_rviz:=false` keeps RViz2 closed.

Gazebo should now contain the simulated UR5e.

RViz2 can still be launched separately if required.

9 ros2_control

`ros2_control` connects ROS 2 controllers to the simulated robot.

The important components are:

- Controller Manager
- Joint State Broadcaster

- Joint Trajectory Controller
- Hardware interfaces

Check the controllers:

```
ros2 control list_controllers
```

The important controllers are:

```
joint_state_broadcaster
scaled_joint_trajectory_controller
```

They should normally show an active state.

9.1 Hardware Interfaces

Check the available interfaces:

```
ros2 control list_hardware_interfaces
```

Typical interfaces include:

- Position
- Velocity
- Effort

10 Command the Simulated UR5e

The trajectory controller accepts:

```
trajectory_msgs/msg/JointTrajectory
```

The command topic is:

```
/scaled_joint_trajectory_controller/joint_trajectory
```

Check it:

```
ros2 topic info \
/scaled_joint_trajectory_controller/joint_trajectory -v
```

10.1 Test From the ROS 2 CLI

Before using Python, test the controller directly:

```
ros2 topic pub --once \
/scaled_joint_trajectory_controller/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['shoulder_pan_joint',
'shoulder_lift_joint',
'elbow_joint',
'wrist_1_joint',
'wrist_2_joint',
'wrist_3_joint'],
points: [{positions: [0.0, -1.57, 0.0, 0.0, 0.0, 0.0],
time_from_start: {sec: 2}}]}"
```

If the setup is correct, the simulated UR5e moves in Gazebo.

This test is useful because it checks Gazebo and the controller without involving the Python node.

11 Python Trajectory Publisher

The same command can be sent from a Python node.

Import:

```
from trajectory_msgs.msg import JointTrajectory
from trajectory_msgs.msg import JointTrajectoryPoint
```

Create the publisher:

```
self.publisher = self.create_publisher(
    JointTrajectory,
    '/scaled_joint_trajectory_controller/joint_trajectory',
    qos
)
```

Create the trajectory:

```
msg = JointTrajectory()

msg.joint_names = [
    'shoulder_pan_joint',
    'shoulder_lift_joint',
    'elbow_joint',
    'wrist_1_joint',
    'wrist_2_joint',
    'wrist_3_joint'
]

point = JointTrajectoryPoint()

point.positions = [
    0.0,
    -1.57,
    0.0,
    0.0,
    0.0,
    0.0
]

point.time_from_start.sec = 2

msg.points.append(point)

self.publisher.publish(msg)
```

11.1 Using a Timer

A timer can trigger the trajectory publisher:

```
self.timer = self.create_timer(
```

```

2.0,
self.publish_joint_trajectory
)

```

The timer period and trajectory duration are different.

- The timer controls when the callback runs.
- `time_from_start` controls when the controller reaches the trajectory point.

For a one-shot command:

```

def publish_joint_trajectory(self):

    self.publisher.publish(msg)
    self.timer.cancel()

```

The node must remain alive until the callback executes, so `rclpy.spin(node)` is required.

12 Verification

Check whether the trajectory is being published:

```

ros2 topic echo \
/scaled_joint_trajectory_controller/joint_trajectory

```

Check joint states:

```

ros2 topic echo /joint_states

```

Check controllers:

```

ros2 control list_controllers

```

Check the TF chain:

```

ros2 run tf2_ros tf2_echo base_link tool0

```

13 RViz2 vs Gazebo

The two stages have different purposes.

	RViz2	Gazebo Harmonic
Purpose	Visualisation	Simulation
Robot description	URDF/Xacro	Robot description + simulation configuration
Physics	No	Yes
Simulated joints	No	Yes
TF2	Yes	Yes
ROS 2 nodes	Yes	Yes
Joint states	ROS-side state	Simulated state
<code>ros2_control</code>	Not required for basic setup	Main control interface
Controllers	Not required for basic visualisation	Used to control simulated joints

Trajectory commands	Not the main focus	Used to move the robot
Main visualisation	RViz2	Gazebo

14 Debugging

Debug according to which stage is failing.

14.1 RViz2 Problems

Check the robot nodes:

```
ros2 node list
```

Check joint states:

```
ros2 topic echo /joint_states
```

Check TF:

```
ros2 run tf2_ros tf2_echo base_link tool0
```

Check the TF tree:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

If these do not work, fix the RViz2/ROS setup before moving to Gazebo.

14.2 Gazebo Problems

First check the controllers:

```
ros2 control list_controllers
```

Then check the trajectory topic:

```
ros2 topic list | grep trajectory
```

Check whether a trajectory is being published:

```
ros2 topic echo \
/scaled_joint_trajectory_controller/joint_trajectory
```

Finally check:

```
ros2 topic echo /joint_states
```

The debugging path is:

Python Node → ROS 2 Topic → Controller → ros2_control → Gazebo → Simulated UR5e

15 Useful ROS 2 Commands

Command	Purpose
<code>ros2 node list</code>	List running nodes.
<code>ros2 topic list</code>	List available topics.
<code>ros2 topic echo /topic</code>	Display messages on a topic.
<code>ros2 topic info /topic -v</code>	Display topic and QoS information.
<code>ros2 interface show ...</code>	Display a message definition.
<code>ros2 control list_controllers</code>	Display loaded controllers.
<code>ros2 control</code>	Display hardware interfaces.
<code>list_hardware_interfaces</code>	
<code>ros2 run tf2_ros tf2_echo ...</code>	Display a TF transformation.
<code>ros2 run rqt_tf_tree rqt_tf_tree</code>	Display the TF tree.

16 Final Workflow

The complete implementation follows this order:

1. Install ROS 2 Jazzy and the required packages.
2. Create the ROS 2 workspace.
3. Clone the UR5e description.
4. Build and source the workspace.
5. Launch the UR5e in RViz2.
6. Check the robot model and six joints.
7. Check joint states.
8. Check TF2 and the TF tree.
9. Create and test the ROS 2 Python node.
10. Understand publishers, subscribers, callbacks, and QoS.
11. Confirm that the ROS 2 setup works in RViz2.
12. Launch the UR5e in Gazebo Harmonic.
13. Check `ros2_control` and its controllers.
14. Send a trajectory from the ROS 2 CLI.
15. Send the trajectory from Python.
16. Verify the complete ROS 2–Gazebo communication chain.

The key approach is:

Build and verify the ROS 2 side in RViz2 first, then add Gazebo simulation and control.